

Vigil

Cómo funciona el agente, explicado desde cero — esquema de trabajo, base de datos y glosario.

Proyecto Mitumi BackStage · Documento de referencia para personas que empiezan · Actualizado: 10/07/2026

■ Para quién es este documento

Está escrito para alguien que **no ha programado** este proyecto (o no ha programado nunca). No hace falta saber nada previo: cada término técnico se explica la primera vez que aparece, y al final hay un glosario. Si solo quieres una cosa: **el histórico de concursos se guarda en la tabla** concursos del fichero `vigil.db` (lo explicamos en la sección 8), y **la API que lee la plataforma está en** `api.py` (sección 7).

Contenido

1. Qué es Vigil, en una frase
2. Las 6 palabras que conviene entender primero
3. El recorrido de una mañana (diagrama)
4. El mismo recorrido, paso a paso
5. La regla de oro: el LLM solo filtra; el código calcula los datos
6. Mapa de archivos: qué hace cada uno
7. La API y el histórico: qué consume la plataforma
8. El esquema de datos (la tabla concursos y los «moldes»)
9. Configuración: modo demo y modo real
10. Qué pasa cuando algo falla
11. Glosario completo

1 · Qué es Vigil, en una frase

Vigil es un **asistente que cada mañana consulta la Plataforma de Contratación Pública de Euskadi (KontratazioA) para las tres diputaciones (Araba, Gipuzkoa y Bizkaia), filtra las licitaciones que le interesan a Mitumi con ayuda de una IA, y publica el resultado hacia la plataforma Mitumi BackStage** (sección «Concursos Públicos»).

La analogía más útil: imagina un **vigilante que cada mañana lee el boletín oficial**, subraya con marcador lo que le puede interesar a Mitumi (una agencia de eventos), anota cuántos días quedan de plazo y deja el resumen sobre la mesa. No se presenta a los concursos, no rellena nada y no manda correos: **observa, filtra y avisa**.

■ Lo que Vigil Sí hace

Lee las convocatorias de las 3 diputaciones, detecta cuáles son **nuevas o modificadas**, las convierte en datos limpios, decide si **encajan con el perfil de Mitumi** (con un motivo y etiquetas temáticas), calcula el **semáforo de urgencia** (días hábiles restantes), guarda un histórico y publica un **JSON** más un **calendario .ics** por concurso para la plataforma.

■ Lo que Vigil NUNCA hace

Presentarse a un concurso ni rellenar ofertas; **enviar correos** (esta versión publica un JSON, no manda emails); modificar la web de contratación (solo la **lee**); ni borrar el histórico. Tampoco deja que la IA invente fechas: los plazos y la urgencia los calcula el código.

2 · Las 6 palabras que conviene entender primero

Palabra	Qué significa aquí (en simple)
Licitación / concurso	Un contrato que una administración pública saca a concurso para que las empresas se presenten (p. ej. organizar un evento institucional). Vigil los vigila; no se presenta.
KontratazioA	La Plataforma de Contratación Pública de Euskadi: la web oficial de donde Vigil lee las convocatorias de las tres diputaciones forales.
Scraping / Playwright	«Leer una web» de forma automática, como haría un navegador manejado por un robot. Vigil usa Playwright para eso (archivo <code>sources.py</code>).
LLM / Groq	La IA que entiende y clasifica texto (modelo Llama servido por Groq). Aquí estructura el texto del pliego y decide relevancia y etiquetas . No calcula fechas ni urgencia.
SQLite / vigil.db	Una base de datos que vive en un solo fichero (<code>vigil.db</code>). Ahí se guarda el histórico de todos los concursos vistos.
Semáforo de urgencia	Un nivel alta / media / baja (o cerrado/desconocida) según los días hábiles que quedan para presentarse. Lo calcula <code>urgency.py</code> .

3 · El recorrido de una mañana (diagrama)

Cuando el agente se ejecuta (automático cada mañana, o a mano con el botón «Actualizar ahora»), atraviesa estos pasos. En cada caja se indica **qué archivo** se encarga.

1

Lee la web de KontratazioA (las 3 diputaciones)

`sources.py` (Playwright)



2

Detecta si cada convocatoria es nueva, modificada o ya vista

`dedupe.py` (SQLite)



3

Convierte cada convocatoria en datos limpios

`extractor.py` (Groq + Pydantic)



4

Decide si encaja con Mitumi + motivo + etiquetas

`relevance.py` (LLM)



5

Calcula el semáforo de urgencia (días hábiles)

`urgency.py`



6

Genera el calendario .ics de cada concurso

calendar_ics.py



7

Guarda TODO en el histórico (relevante o no)

history.py (SQLite · vigil.db)



8

Escribe el JSON y los .ics en la carpeta de salida

publisher.py → salida/



9

La plataforma lo lee del JSON o lo pide por la API

api.py · /concursos · /ejecuciones

■ La clave del recorrido

Solo los pasos **3 y 4** usan la IA (estructurar y decidir relevancia). Todo lo demás —leer, detectar cambios, calcular urgencia, guardar y publicar— es **código normal**. Y el histórico (paso 7) guarda **todos** los concursos, sean relevantes o no. Orquesta el proceso `main.py`.

4 · El mismo recorrido, paso a paso

Paso	Qué ocurre	Archivo	Nota para principiante
1	Se leen las convocatorias de la web oficial.	sources.py	En demo, en vez de la web se usan convocatorias de ejemplo de <code>examples/</code> .
2	Se comparan con lo ya visto.	dedupe.py	Marca cada una como nueva, modificación (p. ej. ampliaron plazo) o ya vista.
3	Se estructura el texto en datos.	extractor.py	La IA saca objeto, órgano, importe, plazo, enlace... en un formato limpio.
4	Se decide si le interesa a Mitumi.	relevance.py	La IA compara con el perfil de Mitumi y da un motivo y etiquetas (Cultura, Gastronomía...).
5	Se calcula la urgencia.	urgency.py	Cuenta los días hábiles que quedan y pone el semáforo. Es código, no IA.
6	Se crea el calendario.	calendar_ics.py	Un archivo .ics por concurso para añadir el plazo a la agenda.
7	Se guarda en el histórico.	history.py	Todos los concursos (relevantes o no) van a la tabla concursos de <code>vigil.db</code> .
8	Se publica la salida.	publisher.py	Escribe concursos.json (los relevantes del día) y los .ics en <code>salida/</code> .
9	La plataforma lo consume.	api.py	Lee el JSON o consulta el histórico y lanza ejecuciones por la API.

5 · La regla de oro: el LLM solo filtra; el código calcula los datos

Es el punto de diseño más importante. Mucha gente imagina que «la IA lo hace todo». **Aquí no.** La IA (`extractor.py` y `relevance.py`) se encarga de dos cosas: **ordenar el texto** del pliego y **decidir si encaja** con Mitumi, con su motivo y etiquetas.

Todo lo que tiene que ser **exacto** lo hace código normal, sin IA:

Fechas y urgencia	Los días hábiles restantes y el semáforo (alta/media/baja) los calcula <code>urgency.py</code> a partir del plazo. La IA nunca inventa una fecha.
Nuevo o modificado	Comparar con lo ya visto y detectar cambios de plazo lo hace <code>dedupe.py</code> contra la base de datos.
El histórico	Qué se guarda y cómo se guarda lo decide <code>history.py</code> , no la IA.

■ Por qué se diseñó así

Porque las decisiones que de verdad importan —¿sigue en plazo?, ¿es nuevo o ya avisamos?— tienen que ser **exactas y repetibles**, aunque la IA fallara o se inventara algo. La IA es un «clasificador» que ayuda a filtrar; no es quien lleva las cuentas. Además, Vigil **no envía correos ni se presenta a nada**: solo lee y publica un JSON que la plataforma recoge.

6 · Mapa de archivos: qué hace cada uno

Agrupados por su papel en el recorrido. Todos viven dentro de la carpeta `vigil/`.

Lectura	<code>sources.py</code> — lee KontratazioA con Playwright. <code>examples/</code> — convocatorias de ejemplo para la demo.
Procesado	<code>dedupe.py</code> — nuevo / modificado / ya visto. <code>extractor.py</code> — estructura el texto (IA). <code>relevance.py</code> — relevancia + etiquetas (IA). <code>urgency.py</code> — semáforo. <code>calendar_ics.py</code> — el .ics.
Memoria	<code>history.py</code> — el histórico en SQLite (<code>vigil.db</code>). Guarda todos los concursos vistos.
Salida	<code>publisher.py</code> — escribe concursos.json y los .ics en <code>salida/</code> para la plataforma.
API	<code>api.py</code> — lo que consume la plataforma: histórico + disparador de ejecución (FastAPI).
Orquestación	<code>main.py</code> — encadena todo el proceso de principio a fin.
Contexto de negocio	<code>business_profile.py</code> — el perfil fijo de Mitumi (qué eventos organiza) que usa el filtro de relevancia.
Moldes de datos	<code>schemas.py</code> — las «fichas» (Convocatoria, VeredictoRelevancia, Urgencia, Alerta).
Config y demo	<code>config.py</code> — ajustes. <code>demo.py</code> — el modo sin IA. <code>serve_demo.py</code> — arranca la demo (puerto 8000).

7 · La API y el histórico: qué consume la plataforma

■ Respuesta directa

La plataforma habla con Vigil a través de `api.py` (FastAPI). Puede **consultar el histórico** o **lanzar una ejecución en vivo** (el botón «Actualizar ahora»). El histórico persiste en `vigil.db`; el seguimiento de ejecuciones se guarda en memoria (se reinicia con el servidor).

GET /concursos	Consulta el histórico con filtros: <code>q</code> (texto), <code>diputacion</code> , <code>urgencia</code> , <code>en_plazo</code> , <code>relevante</code> , <code>limite</code> , <code>offset</code> .
POST /ejecuciones	Lanza el agente en vivo (scrape + IA) en segundo plano. Devuelve un <code>id</code> y un estado.
GET /ejecuciones/{id}	El estado de esa ejecución (<code>en_curso</code> / <code>terminada</code> / <code>error</code>) y cuántos concursos nuevos añadió — para hacer <i>polling</i> desde la web.
GET /concursos/{id}/calendario.ics	Descarga el calendario de un concurso concreto.
GET /health	Sonda de vida (comprobar que la API responde).

■ El contrato de salida

Además de la API, cada ejecución escribe en la carpeta `salida/`: `concursos.json` (los concursos relevantes del día, cada uno con título, organismo, fechas, importe, urgencia, etiquetas, si es modificación y su `.ics`) y `ics/*.ics` (un calendario por concurso). La carpeta se cambia con `VIGIL_OUTPUT_DIR`.

8 · El esquema de datos (la tabla concursos y los «moldes»)

El histórico vive en una tabla de SQLite llamada `concursos` (en `vigil.db`). Su identificador único es el `id_expediente`. Estas son sus columnas:

id_expediente	Código único del expediente (la clave primaria).
diputacion · objeto · organo_convocante	De qué diputación es, qué se pide y quién convoca.
importe · plazo_presentacion · plazo_iso	Importe, fecha límite en texto y esa misma fecha en formato ISO (para ordenar y filtrar «en plazo»).
enlace_pliego · fecha_publicacion · fecha_ultima_publicacion	Enlace al pliego y fechas de publicación (la última sirve para detectar modificaciones).
relevante · motivo · etiquetas · campos_no_verificables	Si encaja con Mitumi (1/0), el porqué, las etiquetas temáticas y los requisitos que no se pudieron verificar.
urgencia_nivel · urgencia_dias	El semáforo (alta/media/baja...) y los días hábiles restantes.
visto_por_primera_vez · ultima_actualizacion	Cuándo se vio por primera vez y cuándo se actualizó por última vez.

En el código, esos datos se manejan con «moldes» de Pydantic (en `schemas.py`): **Convocatoria** (la convocatoria limpia), **VeredictoRelevancia** (relevante, motivo, etiquetas), **Urgencia** (nivel, días, etiqueta) y **Alerta**, que junta las tres más si es una modificación.

9 · Configuración: modo demo y modo real

VIGIL_DEMO=1	Activa el modo demo : usa las convocatorias de <code>examples/</code> , salta la IA (<code>demo.py</code>) y usa base y salida aparte (<code>vigil_demo.db</code> , <code>salida_demo/</code>). No toca datos reales, pero recorre el mismo pipeline. Web en el puerto 8000.
Modo real	Quita <code>VIGIL_DEMO</code> , define <code>GROQ_API_KEY</code> (la IA) y ejecuta <code>playwright install chromium</code> (el navegador para leer la web).
VIGIL_OUTPUT_DIR	Carpeta donde se escribe la salida (por defecto <code>salida/</code>).
Automatización diaria	<code>.github/workflows/vigil.yml</code> ejecuta el agente cada mañana con GitHub Actions (solo si el proyecto está en la raíz de su repositorio).
Dependencias	<code>fastapi/uvicorn</code> (API), <code>pydantic</code> (moldes), <code>playwright</code> (scraping) y el cliente de <code>groq</code> (IA).

10 · Qué pasa cuando algo falla

Falta la clave de la IA (modo real)	El filtro de relevancia no puede trabajar. Para enseñar el agente sin clave se usa el modo demo, que salta la IA.
La web oficial no responde o cambia	El scraping puede no traer convocatorias ese día; el resto del proceso no se rompe, simplemente no hay novedades.
Un concurso sin plazo claro	La urgencia queda como «desconocida» en vez de inventarse un número. No rompe el proceso.
Un concurso ya visto	<code>dedupe.py</code> lo reconoce: lo marca como «ya visto» o «modificación», y no lo duplica en el histórico.
Se reinicia el servidor	El histórico persiste (está en <code>vigil.db</code>), pero el seguimiento de ejecuciones en curso (que vive en memoria) se pierde.

Glosario · completo

Licitación / concurso público

Un contrato que una administración saca a concurso para que las empresas presenten ofertas.

KontratazioA

La Plataforma de Contratación Pública de Euskadi, la web oficial de donde Vigil lee.

Scraping / Playwright

Leer una web de forma automática, como un navegador manejado por un robot. Playwright es la herramienta.

LLM / Groq

La IA que clasifica texto (modelo Llama servido por Groq). Aquí estructura y filtra; no calcula fechas.

SQLite / vigil.db

Una base de datos que cabe en un solo fichero. Guarda el histórico de concursos.

.ics

Un archivo de calendario estándar; sirve para añadir el plazo de un concurso a la agenda.

Semáforo de urgencia

Nivel alta/media/baja según los días hábiles que quedan de plazo.

Días hábiles

Días laborables (sin fines de semana ni festivos) que se cuentan para el plazo.

Dedupe

«Des-duplicar»: reconocer si un concurso ya se vio o si cambió, para no avisar dos veces.

Histórico

El registro de todos los concursos vistos (relevantes o no), guardado en la base de datos.

Relevancia

La decisión de si un concurso encaja con lo que hace Mitumi, con un motivo explicado.

Etiquetas

Temas que la IA asigna (Institucional, Cultura, Gastronomía...) para repartir la revisión.

JSON

Un formato de texto para intercambiar datos entre programas. Vigil publica un `concursos.json`.

Polling

Preguntar cada poco «¿ya terminó?» — la web lo hace para saber el estado de una ejecución.

FastAPI / uvicorn

La herramienta con la que está hecha la API y el servidor que la pone en marcha.

Documento de apoyo del agente Vigil — Mitumi BackStage. Refleja el estado del código a 10/07/2026. Para el detalle técnico exacto, la fuente de verdad son los archivos del propio proyecto (`README.md` y el código en `vigil/`).